Network Models

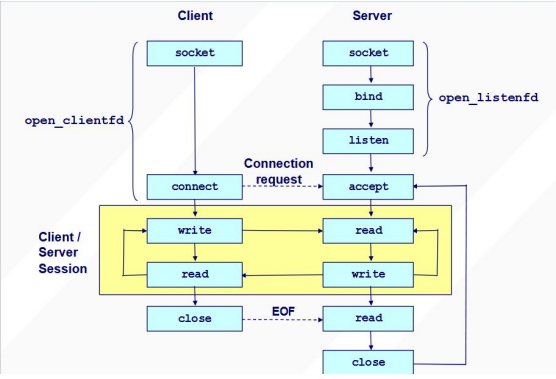
OSI Model	TCP/IP
7. Application	Application
6. Presentation	
5. Session	
4. Transport	Transport
3. Network	Network
2. Data link	Link
1. Physical	

Data wrapped with headers at each layer (HTTP -> TCP -> IP -> Ethernet

Raw IP is not used due to unreliability, TCP/UDP abstracts complexities for apps

TCP (Transmission Control Protocol)	UDP (User Datagram Protocol)
Connection-oriented (3-way handshake: SYN -> SYN-ACK -> ACK)	Connectionless (no handshake, packets treated separately and independently)
Reliable (retransmits lost packets)	Unreliable (no retransmission)
In-order Delivery	No ordering guarantee
Higher latency, more overhead	Lower latency, less overhead
Stream-based (data as a continuous byte stream, no message boundaries)	Datagram-based (packet boundaries)
Web (HTTP), email, file transfer	DNS, VoIP, streaming

Socket: An endpoint for communication between machines (IP + Port)



Performance Metrics

Bandwidth: # bits transmitted per unit time in a channel

Latency: Propagation+Transmit+Queue

- Propagation=Distance/Speedof Light(*)
- Transmit = packet bits/Bandwidth
- Queue = Time waiting in traffic jams

Overhead: # secs for CPU to transmit

Error Rate: Probability p message will not arrive intact

10 MB Object

	Propagation: 1 ms	Propagation: 100 ms
Bandwidth: 1 Mbps	80.001 s	80.1 s
Bandwidth: 100 Mbps	.801 s	.9 s

Best-Effort Service

- No guarantees, lost, duplicated, reordered, delayed, etc
- Packet Sizes up to 64KB (typically 1500 bytes)

IP Packet Header

Fields: Version, TTL, Protocol (TCP/UDP), Source/Dest IP, Checksum

CIDR Notation

192.168.1.0/24: 24 bits for network, 8 for host
Allows for flexible allocation (/19 = 13 host bits)

Subnet Mask

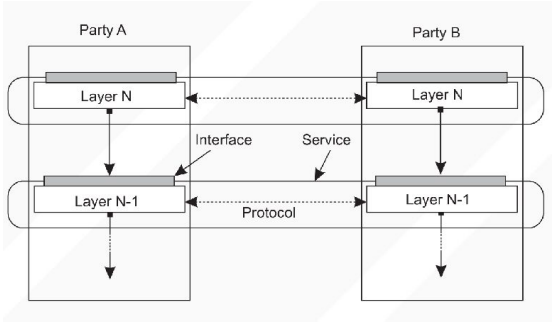
/24 = 255.255.255.0
Used to define network bits

Aggregation

Reduces router table size
(e.g., 212.56.132.0/22 covers four /24 subnets, 4 subnets share /22 host bits)

Protocols

Rules for communication between similar layers



Service Interface: what operations available to users
Protocol interface: how operations are encoded on the wire (formatting of the messages)

Protocols are specified in:
Prose/BNF, Message Sequence Diagrams, State Transition Diagrams, Packet Formats

Computer architecture: **Mega 2^20, Kilo 2^10**
Computer networks: **Mega 10^6, Kilo 10^3**

Mbps vs. MBps
Networks: megabits per second
Architecture: megabytes per second

Bandwidth vs. Throughput

- Bandwidth: available over link
- Throughput: available to application

Socket Blocking & Deadlock

- send(sock, buff, n, 0) blocks if:
  - N > SQS -> blocks until n-SQS bytes move to RecvQ
  - N > SQS + RQS -> blocks until receiver recv()'s n-(SQS +RQS) bytes
- Deadlock risk:
  - Both sides call recv() but neither sends -> both block waiting for data
- Avoid deadlock:
  - Ensure at least one side sends data
  - Don't send more than buffers can hold without letting receiver read

Performance Tools

Ping: Measures round-trip latency
Iperf3: Tests TCP/UDP Throughput

Addressing

IPv4:

- 32-bit address
- (192.168.1.10)
- Four 8-bit octets

IPv6:

- 128-bit address
- (2001:0DB8:AC10:FE01::)

127.0.0.1: Loopback (local machine)
Host bits 0: Network address
Host bits 1: Broadcast address
Useable Address Formula:

$$2^{32-n} - 2$$

Port Numbers

0-1024: Reserved for OS
1025-65535: Available

DNS:

Converts domain names to IP addresses
net.LookupIP("example.com")

Framing

Writing out (and reading in) messages from a stream such that messages can be separated and interpreted correctly
Choices:

1. *Length in each segment (security concerns about reading some random length)*
2. *Delim (escape mech if the delimiter is within the message)*

Parsing & en-/de- coding

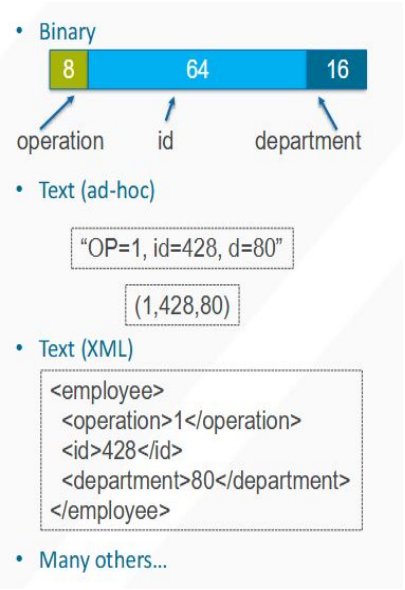
Converting a message to/from an app-level data structure

Operation

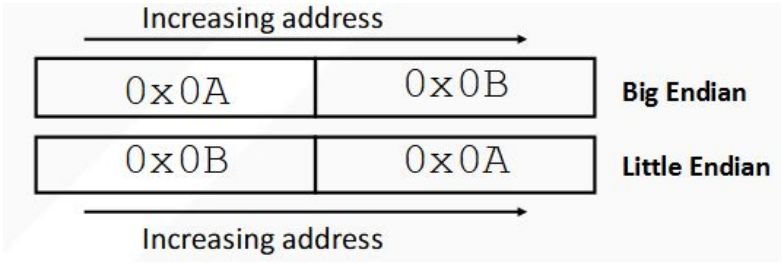
Action performed within protocol's service interface

Message

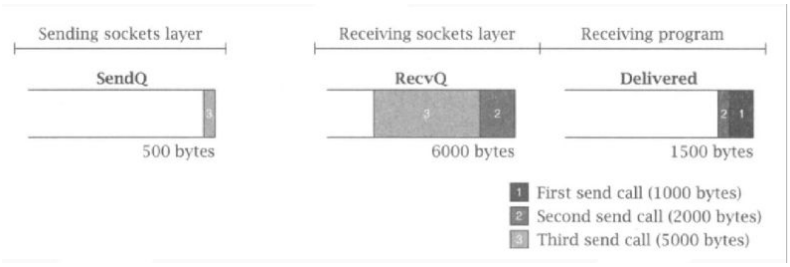
Encoding of an operation or data according to protocols *wire format.*



Endianess



For internet, we standardize on big-endianess



Example TCP Golang connection

```
// Dial TCP (client)
conn, err := net.Dial("tcp",
"localhost:8080")
conn.Write([]byte("hello"))
buf := make([]byte, 1024)
n, _ := conn.Read(buf)

// Listen TCP (server)
ln, err := net.Listen("tcp", ":8080")
for {
        conn, _ := ln.Accept()
        go handleConn(conn)
}
func handleConn(c net.Conn) {
        defer c.Close()
        buf := make([]byte, 1024)
        n, _ := c.Read(buf)
        c.Write([]byte("ack"))
}
```

```
// Main loop of a server
Remaining := ""
buf := make([]byte, 1024)
for {
        for "Does remaining contain a full
        request?""
        {
                If yes,(1) parse it, then (2) remove
                from remaining
        }
        size, err := c.Read(buf)
        data := buf[:size]
        remaining = remaining + string(data)
}
```

Tail Latency

High-percentile delays in distributed systems (small amount of servers take a long time) overall latency delayed by the slowest response

Aggregation Server / Front-end Server

Server that acts as “well-known” server to interact with clients, does the heavy lifting of searching index servers for search terms.

Large-scale online services: parallelize sub-operations, fan out request from root to leaf servers, then merge responses with request-distribution tree

With N servers, P(slow) per server

$$P(\text{any slow}) = 1 - (1 - P)^N$$

(grows with more and more servers)

Percentile: p50 = median, p99 - wirst 1%; reduce 99% latency with techniques: hedging, reducing variability, and canary requests

Amplification: 5% slow servers -> 50% latency impact

Variability is dynamic: mitigation is better than eliminating slow servers

Hardware trends: tolerating variability more important over time; higher bandwidth reduces per-message overheads (reduces costs of tied req.)

Reducing component variability:

- **Differentiating service classes**
  - Differentiate non-interactive requests
  - Prioritize interactive over background tasks
- **High level queuing:** keep low level queues short
- **Reduce head-of-line blocking**
  - Break long-running requests into a sequence of smaller requests (request partitioning)
- **Synchronize disruption**
  - Do background activities altogether

Short-term (request) mitigation strategies:

- **Hedged Requests:** send duplicate request after a certain threshold, accept first response back
- **Tied requests:** send to multiple servers with cross-cancellation; once a server picks up the request, cancel the tied counterparts

Long-term adaptation mitigation strategies:

- **Load shedding:** temporarily de-prioritize slow nodes when a server exceeds a latency threshold; remove from being active, continue to send shadow traffic to monitor recovery
- **Micro-partitions:** break data into multiple partitions to quickly redistribute across servers, process in parallel
- **Selective replication:** replicate and distribute items that are detected/predicted to cause load imbalance
- **Canary Analysis:** route untested/new code paths to a small amount of leaf servers, only continue querying rest of servers if canaries returned within threshold